Security Engineering
OCL Tutorial

Srđan Krstić Daniel Galán Hoang Nguyen
Institute of Information Security
Department of Computer Science
ETH Zürich
srdan.krstic@inf.ethz.ch

Contents

1 Introduction to OCL

Object Constraint Language (OCL) is a language for specifying constraints and queries using a textual notation. It is a part of the Unified Modeling Language (UML): in Version 1.1 Specification, the OCL appears as the standard for specifying invariants, pre- and post-conditions; however, as in Version 2.0 Specification, the OCL has been assigned for a broader use, including usage in the definition of specific domain metamodels, model transformation, model testing and validation.

OCL is a strongly-typed language: each expression has either a primitive type, a class type, a tuple type, or a collection type. Collections can be sets, bags, ordered sets and sequences, and can be parametrized by any type, including other collection types. The language provides standard operators on primitive types, tuples, and collections. Every OCL expression is written in the context of a data model (the so-called *contextual* model).

1.1 Data model: Application domain

A data model represents the state space of your application and the data representation that your application will manipulate. For example, if you develop an online flight booking application, you might include in your domain model objects like *Person*, *Flight*, *Booking* etc.

A good practice is to model the data model of an application independently of the application logic or user interface. This approach leads to classes with almost no logic and a lot of attributes and relations, e.g., a *Person* class might have the *firstName*, *lastName*, and *address* attributes.

1.2 Syntax & Semantics

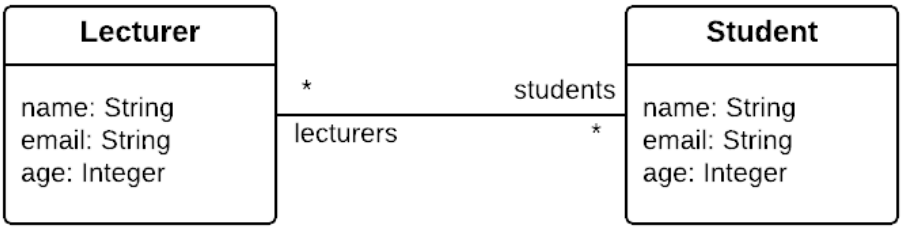
For objects, OCL provides a notational style similar to that of object-oriented languages: a dot-operator to access the value of an attribute of the object, or the collection of objects linked with another object at the end of an association. For example, suppose that the contextual model includes a class c with an attribute at and an association-end ase. Then, if o is an object of the class c, in the given data model instance, the expression $o\,.\,at$ refers to the value of the attribute at of the object o, and $o\,.\,ase$ refers to the collection of objects linked to the object o at the association-end ase.

For collections, OCL provides an `allInstances`-operator to collect all objects of a specific class and an arrow-operator "→" to either access a property of the collection or to iterate over the collection and perform some actions. For example, suppose that the contextual model includes a class c. Then, c.`allInstances()` represents the collection of all objects in class c. Now, suppose that *source* represents a collection. Then, *source*→`size()` returns the size of this collection, *source*→`isEmpty()` returns whether this collection is empty, *source*→`forAll(`$v|$*body*`)` iterates over this collection and checks whether all elements v in this collection

satisfy the property stated in *body*, *source*→`exists(`*v*|*body*`)` iterates over this collection and checks whether there exists at least one element v in this collection satisfies the property stated in *body* and *source*→`includes(`*o*`)` iterates over this collection and checks whether object o is included.

Example

Consider the simple university model as the underlying data model of the example.



To know the number of students, in OCL, one can express as follows:

$$\underset{(1)}{\texttt{Student.allInstances()}}\underset{(2)}{\texttt{→size()}}$$

in which (1) is a subexpression that applies the `allInstances`-operator on the `Student` class, and (2) is an arrow-operator that applies the *size* property on (1).

To know whether there is a student that is taught by no lecturer, in OCL, one can express as follows:

$$\underset{(1)}{\texttt{Student.allInstances()}}\underset{(2)}{\texttt{→exists(s|}}\underset{(3)}{\texttt{s.lecturers→ size() = 0}}\texttt{)}$$

in which (1) is as above, and (2) is an operator that iterates over (1) and checks whether any student `s` has a number of lecturers that is equal to 0 — i.e. `s` has no lecturer — which is precisely defined in (3).

To know whether every lecturer teaches every student, in OCL, one can express as follows:

$$\underset{(1)}{\texttt{Student.allInstances()}}\underset{(2)}{\texttt{→forAll(s|}}$$
$$\underset{(3)}{\texttt{Lecturer.allInstances()}}\underset{(4)}{\texttt{→forAll(l|}}\underset{(5)}{\texttt{s.lecturers → includes(l)}}\texttt{))}$$

in which (1) and (3) are sub-expressions that apply the `allInstances`-operator on the `Student` and `Lecturer` class, respectively; (2) is an operator that iterates

over (1) and checks whether for every student `s` in (1), `s` satisfies that, for every lecturer `l` in (3), `l` is a lecturer of `s`, which is precisely defined in (5).

See the `Additional Resources` tab on the course Moodle page for a more complete OCL language reference.

2 OCL-py: A Python OCL evalution library

This section shows how to create a simple data model in Python and use the OCL-py library to write and evaluate Object Constraint Language (OCL) queries and constraints on an instance of the data model.

As a running example for this tutorial, we consider a simple data domain: Web (as a collection of webpages and categorized articles).

2.1 Installation

The easiest way to get started is to use Docker. Assuming you have Docker installed, you can run the following command to start a container with OCL-py:

```
$  docker run -it -v .:/app/test infsec/ocl-py
```

You will get access to a terminal inside the container where you can run Python scripts using OCL-py. The current working directory will be mounted to the container at `/app/test`, so you can create and edit your Python files there from your host machine. To test the installation, you can run the following command:

```
(.venv) python
>>> from ocl.ocl import eval_ocl
>>> eval_ocl("1+1")
2
```

2.2 Data model: Webpage Application

Now let's create our example domain model in Python. The corresponding UML class diagram should look like this:

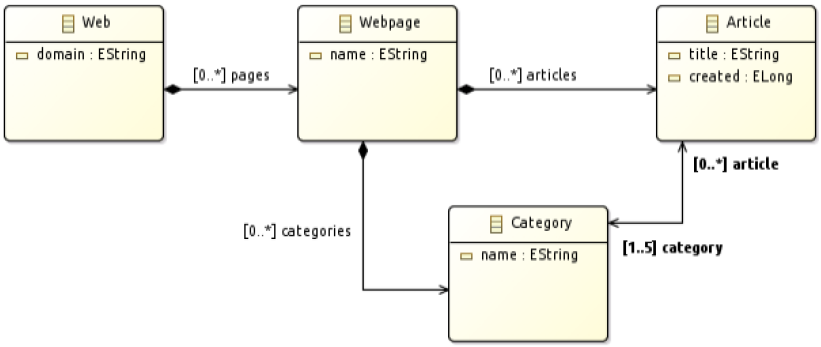


Figure 1: Example domain model

You can use the provided `model.py` file:

```python
from ocl.ocl import OCLTerm

class Web(OCLTerm):
    def __init__(self, domain):
        self.domain = domain
        self.pages = set()

    def add_page(self, page):
        self.pages.add(page)
        page.web = self

    def __repr__(self):
        return f"Web(domain={self.domain})"

class Webpage(OCLTerm):
    def __init__(self, name, web):
        self.name = name
        self.categories = set()
        self.articles = set()
        self.web = web
        web.add_page(self)

    def __repr__(self):
        return f"Webpage(name={self.name}, web={self.web.domain})"

    def add_category(self, category):
        self.categories.add(category)
        category.webpage = self

    def add_article(self, article):
        self.articles.add(article)
        article.webpage = self

class Category(OCLTerm):
    def __init__(self, name, webpage):
        self.name = name
        self.webpage = webpage
        webpage.add_category(self)
        self.articles = set()

    def add_article(self, article):
        self.articles.add(article)
        article.category.add(self)

    def __repr__(self):
        return f"Category(name={self.name}, webpage={self.webpage.name})"
```

```python

class Article(OCLTerm):
    def __init__(self, title, webpage):
        self.title = title
        self.webpage = webpage
        webpage.add_article(self)
        self.category = set()

    def add_category(self, category):
        self.category.add(category)
        category.add_article(self)

    def __repr__(self):
        return f"Article(title={self.title}, webpage={self.webpage.name})"
```

2.3 Object model: Instantiate the data model

Instantiate the objects and relations as shown in the following object diagram (with instances of the bi-directional "category-article" reference shown in red):

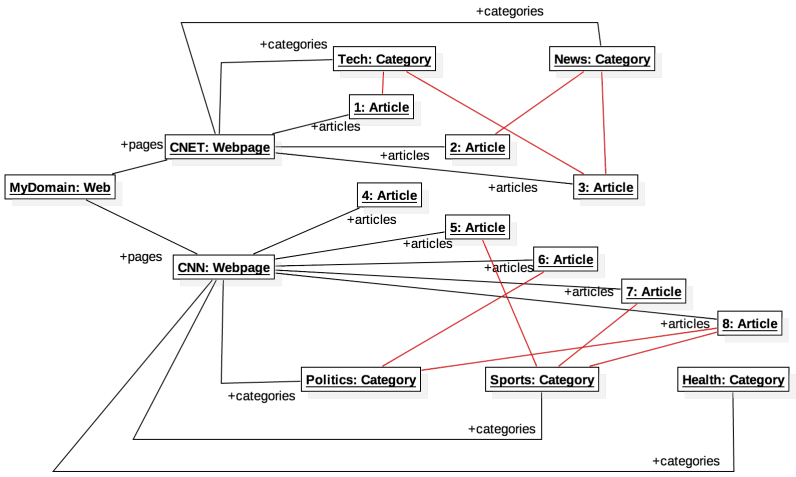


Figure 2: My.web object diagram.

Webpage = {CNET, CNN}, Category = {Tech, News, Politics, Sports, Health}, Article = {1, ... , 8}

The following Python code creates the object model shown in Figure 2:

```python
from ocl.ocl import eval_ocl
from model import Web, Webpage, Category, Article
import model

# Domain, Webpages, Categories, and Articles
d = Web("MyDomain")
p1 = Webpage("CNET", d)
p2 = Webpage("CNN", d)
c1 = Category("Tech", p1)
c2 = Category("News", p1)
c3 = Category("Politics", p2)
c4 = Category("Sports", p2)
c5 = Category("Health", p2)
a1 = Article("Article 1", p1)
a2 = Article("Article 2", p1)
```

```
a3 = Article("Article 3", p1)
a4 = Article("Article 4", p2)
a5 = Article("Article 5", p2)
a6 = Article("Article 6", p2)
a7 = Article("Article 7", p2)
a8 = Article("Article 8", p2)

# Adding articles to categories
c1.add_article(a1)
c1.add_article(a3)
c2.add_article(a2)
c2.add_article(a3)
c3.add_article(a6)
c3.add_article(a8)
c4.add_article(a5)
c4.add_article(a7)
c4.add_article(a8)
```

3 Exercise: OCL queries

The Object Constraint Language is a formal language that expresses constraints or object query expressions on a UML model. See the `Additional Resources` tab on the course Moodle page for a more complete OCL language reference. In order to execute OCL on our object model, you can use the `eval_ocl` function (as shown in Section 2.1).

Task: Practice OCL

Formulate and evaluate the following queries in OCL:
Q1) "The context object"
Q2) "All webpages of the context object"
Q3) "Names of all webpages of the context object"
Q4) "Set of names of all webpages of the context object"
Q5) "Set of names of all webpages"
Q6) "Number of categories"
Q7) "All categories with no articles classified"
Q8) "All articles classified according to more than one category"
Q9) "All articles that are in Politics or News category"
Q10) "All articles that are both in Tech and News category"

Also, formulate and evaluate the following constraints:
C1) "Article can be part on only one Webpage"
C2) "Any article from a webpage W can only be classified according to categories from W.categories"

.....

The solution will be published in Moodle as `ocl-solution`.

Using `eval_ocl()` under contextual model

When writing OCL expressions that contain free variables, e.g.,

```
self.age > x
```

The `eval_ocl()` function must map the free variables used in the given OCL expression to current Python instances. You must provide these variables as keyword arguments in the call to `eval_ocl()`. For example, if your OCL expression refers to `self`, you must pass it explicitly. As an example,

```
eval_ocl("self.age > x", self=p, x=18)
```

Here `self` is bound to the Python person object `p`, and 18 is assigned to `x`.